

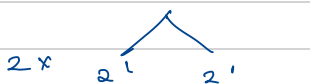
Q1 Power function, $a \in n$ find a^n

```
⇒ int power (int a, int n) {  
    // base  
    if (n == 0) return 1;  
  
    // normal case  
    return a * power(a, n-1);  
}
```

Optimisation

$$2^6 = \overbrace{2 \times 2 \times 2}^2 \times \overbrace{2 \times 2 \times 2}^2$$

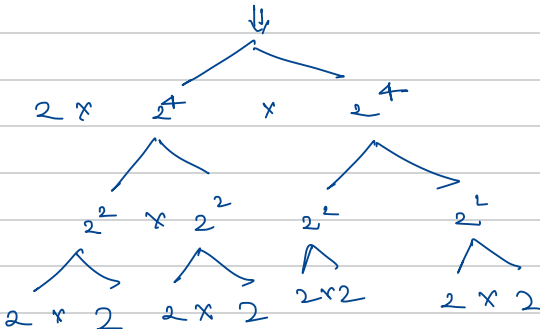
can we like break down the power?

$$2^6 = 2^3 \times 2^3$$


$2 \times 2^1 \times 2^1$

$$2^9 = 2 \times 2^{9/2} \times 2^{9/2}$$

$2^9 \Rightarrow$ odd
 $a \times$



$2 \times 2^4 \times 2^4$

$2^2 \times 2^2 \times 2^2$

$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$

$\Rightarrow$ if power is odd $a^n = a \times \text{power}(a, n/2) \times \text{power}(a, n/2)$
 is even $a^n = \text{power}(a, n/2) \times \text{power}(a, n/2)$

```
long power(int a, int n) {
```

// edge cases

```
if (a == 0) return 0;
```

```
if (a == 1) return 1;
```

// base case

```
if (n == 0) return 1;
```

```
if (n % 2 == 0) {
```

```
    return pow(a, n/2) * pow(a, n/2);
```

```
} else {
```

```
    return a * pow(a, n/2) * pow(a, n/2);
}
```

$\Rightarrow$ optimize further

```
long halfPower = power(a, n/2);
```

```
if (n % 2 == 0) {
```

```
    return halfPower * halfPower;
```

```
} else {
```

```
    return (long) a * halfPower * halfPower;
```

```
}
```

$$a^2 = \text{return } a \times a$$

$$\text{return } a \times \text{pow}(a, 1) \times \text{pow}(a, 1)$$

$$a^3 = \text{return } a \times a \times a$$

$$\text{return } a \times \text{pow}(a, 1) \times \text{pow}(a, 1)$$

DRS ROT

2^5

power (2, 5) { = 32

halfPower = power (2, 2) { = 4

halfPower = power (2, 1) { = 2

halfPower = power (2, 0) { = 1
return 1;

1 % 2 != 0
return 2 * 1 * 1;

2 % 2 == 0
return 2 * 2 = 4

5 % 2 != 0

return 2 * 4 * 4 = 32

